全栈线上实操评测题｜Bid Arena：多人实时拍卖间

您好，感谢您参加本次全栈线上实操评测。

这不是一道考察八股、页面数量或代码行数的题目。我们希望通过一个范围可控的多人实时业务，了解您是否具备真实的工程判断、动手能力，以及使用 Coding Agent 提高交付效率并对最终结果负责的能力。本题允许并鼓励使用 Coding Agent（Codex、Claude Code、Cursor、Copilot 等）：可以让 Agent 搭建结构、生成代码、补测试和协助排错，但方案决策、结果验证和对最终代码的解释与修改必须由您本人负责。未完成的部分请如实写入 README，清楚说明边界比用文档掩盖问题更重要。

本题不涉及真实资金，所有余额、冻结金额与成交款均为测试虚拟积分。

项目目标

实现一个可供真人用户、测试脚本用户与竞拍 Agent 同时参与的多人实时拍卖间。管理员创建并开始限时拍卖，真人用户通过 Vue 页面参与，测试脚本模拟多人并发出价，竞拍 Agent 使用受限 Token 通过公开 API 读取状态并自主决定是否出价。服务端负责身份、余额、冻结金额、最高价、延时规则与唯一结算，客户端与 Agent 不能自行决定最终结果。请优先完成一个小而完整、能稳定验证的产品，不要求复杂运营后台、真实支付、聊天、推荐系统或精细美术。

技术要求

- 前端：Vue 3 + TypeScript（推荐 Vite、Pinia）。必须提供可操作的浏览器界面，不接受仅用 Swagger、Postman 或测试脚本演示业务。
- 后端：Java 17 + Solon 3.x，HTTP API + WebSocket（或等价实时方案）。数据访问方式不限，但必须能解释事务、锁与 SQL 边界。
- 数据：MySQL 8 是用户、钱包、资金流水、拍卖、出价与成交结果的唯一事实来源。数据库迁移与种子数据必须自动执行，评审不手工建表或插入账号。
- 交付：Docker Compose 一键启动 MySQL（推荐含前后端）。Redis 非必选；若使用，必须说明它解决的真实问题，以及与 MySQL 不一致时如何恢复。

两类 Agent

本题涉及两个不同概念：Coding Agent 协助您开发、测试与排查；竞拍 Agent 则是拍卖参与者，持有受限 Agent Token，通过公开 API 查询状态并出价。竞拍 Agent 不能直接访问数据库、管理接口或他人私有数据；客户端与 Agent 也不能代替服务端决定余额、当前最高价、截止时间或赢家。

拍卖与资金规则

拍卖状态机为 DRAFT → RUNNING → SETTLING → FINISHED，另可从 DRAFT 或 RUNNING 进入 CANCELLED；SETTLING 后不可取消。只有 RUNNING 状态接受出价，截止时间以服务端时间为准，到期后产生唯一的最终成交结果。

1. 数值规则：起拍价 100 积分，尚无出价时当前最高价即起拍价；最小加价 10 积分；默认时长 180 秒；测试用户初始余额 1000 积分。新出价不得低于"当前最高价 + 最小加价"。
2. 余额口径：可用余额 = 总余额 − 冻结金额。出价校验的是本次需新增冻结的金额（出价金额减去该用户在本场已冻结金额）不得超过可用余额。
3. 截止边界：出价被接受的条件是服务端接收时间严格早于当前截止时间；恰好等于截止时刻按迟到处理。
4. 幂等与并发：同一 requestId 重复提交只能生效一次，并返回可识别的幂等结果；两个用户并发出价必须得到确定、可解释的唯一赢家。
5. 冻结余额：新出价成为最高价后须释放上一位领先者的冻结金额；同一用户加价只增加差额冻结；竞价失败不得改变任何资金状态。任意时刻余额、可用余额、冻结金额均非负，资金流水可解释；事务失败不得留下"价格已更新但资金未冻结"或相反的半完成状态。出价、资金变动与通知如何编排成一致的整体由您自行设计，并在 DESIGN.md 说明方案、理由与失败分支。
6. 最后五秒延时：截止前 5 秒内的合法出价使截止时间以当前截止时间为基准延长 10 秒；延时判定始终以最新截止时间为准，每场最多延长 3 次，达到上限后出价仍可成功但不再延时。延时必须与该次出价处于同一一致性边界，并实时通知所有参与者。
7. 唯一结算：赢家冻结金额转为实际扣款，其他人的冻结金额全部释放；无人出价时允许无赢家结束，不得产生扣款。到期后系统必须自行完成结算（定时扫描或等价机制），不依赖任何客户端调用，服务重启后必须能继续结算已到期但未结算的拍卖。重复定时任务、重复请求、服务重启或两个实例同时触发结算，均不能重复扣款、重复生成成交记录或重复记流水。
8. 通知边界：WebSocket 广播失败不得回滚已成功提交的数据库事务，客户端应能通过快照重新同步。

数据模型

命名可以不同，但至少覆盖以下事实。并发与重复请求下的正确性不能只依赖应用层"先查询再判断"；采用什么机制保证、它的边界和失效场景是什么，由您决定并需要能解释清楚。

数据	最低要求
用户	身份、角色、密码哈希、状态
钱包	总余额、冻结金额，以及并发更新所需字段（由您设计）
资金流水	用户、类型、金额、关联拍卖、关联请求、创建时间
拍卖	状态、起拍价、当前价、领先者、截止时间、延长次数

数据	最低要求
参与者	拍卖与用户关系、加入时间、Agent / 真人标识
出价	拍卖、用户、金额、requestId、服务端序号、服务端时间
成交结果	拍卖、赢家、成交价、结算原因；一场拍卖最多一条
Agent Token	Token 摘要、所属用户、拍卖范围、权限、过期时间、吊销状态

接口与事件

必须提供 OpenAPI 文档，路径可调整，但需覆盖以下能力：

```
POST /api/v1/auth/login          GET /api/v1/users/me        GET /api/v1/wallets/me
POST /api/v1/admin/auctions      POST /api/v1/admin/auctions/{id}/start
POST /api/v1/admin/auctions/{id}/cancel
POST /api/v1/admin/agent-tokens
GET  /api/v1/auctions            GET  /api/v1/auctions/{id}
POST /api/v1/auctions/{id}/join  POST /api/v1/auctions/{id}/bids
GET  /api/v1/auctions/{id}/bids
```

出价请求至少包含 `requestId` 与 `amount`；所有服务端确认事件必须包含 `auctionId`、单场拍卖内单调递增的 `seq`、`serverTime` 与 `type`（如 `BID_ACCEPTED`）。推荐事件：AUCTION_SNAPSHOT、PARTICIPANT_JOINED、BID_ACCEPTED、BID_REJECTED（只推给本人）、AUCTION_EXTENDED、AUCTION_FINISHED、CONNECTION_STATE。客户端发现序号缺口时，应重新获取权威快照，而不是继续猜测本地状态。

竞拍 Agent Token

管理员可签发竞拍 Agent Token，至少包含或关联：Agent 用户 ID、允许访问的 auctionId、`auction:read` / `auction:bid` 权限、过期时间、吊销状态与可配置的请求频率限制。数据库只保存 Token 摘要，明文仅在创建时返回一次，且不得出现在 Git、前端包、日志或录屏中。Agent API 可以复用业务服务，但必须使用独立的认证凭据与最小权限边界。请提供 `AGENT_TOOL_SPEC.md` 或 OpenAPI 示例，使评审可以将 Token 交给 Coding Agent，由它查询状态并进行拍卖决策；文档中不得包含真实 Token。

多人模拟脚本与自动化测试

仓库必须提供可直接执行的模拟脚本（例如 `npm run simulate:auction -- --auctionId= --bots=20 --seed=42`；如使用 Agent Token，请通过环境变量 `AUCTION_AGENT_TOKEN` 读取，不得写入命令历史或仓库），至少模拟：20 个用户加入同一拍卖间、普通递增出价、两名用户并发提交相同或相邻价格、相同 `requestId` 重试、余额不足 / 金额过低 / 已结束后出价、最后五秒狙击并触发延时、断线后重新获取快照，以及拍卖结束后对赢家、成交价、余额、冻结金额与成交记录的核对。脚本必须输出断言结果和最终摘要；正式评审会使用独立隐藏脚本按公开规则进行黑盒验证。

自动化测试至少覆盖：非法状态、低价、余额不足的出价被拒绝；同一 `requestId` 只产生一次出价和一次资金变化；两个用户并发出价后只有一个确定领先者；新领先者冻结、旧领先者释放与同一用户加价差额正确；最后五秒延长且最多三次；到期、取消或重复触发结算只生成一个成交结果；结算后余额、冻结金额和流水一致；Agent Token 的范围、权限、过期和吊销生效；WebSocket 推送包含递增 `seq` 且重连可恢复快照；至少一个 Vue Store 或核心交互组件测试。关键并发与结算测试必须使用真实 MySQL 或等价的容器化集成环境，不能全部由内存 Mock 代替。

快速启动与初始数据

请把"评审五分钟内跑起来"视为交付的一部分。根目录必须提供 `.env.example`、`docker-compose.yml`（或 `compose.yaml`）、数据库 migration、自动种子命令或启动时自动初始化、一键测试命令与多人模拟脚本。建议提供以下演示账号（如有调整必须在 README 明确）；种子数据至少包含一件可由管理员立即开始的演示拍品，不要让拍卖在服务启动时自动倒计时：

身份	账号	密码
管理员	`admin@example.com`	`Admin123456!`
真人 A	`bidder_a@example.com`	`Test123456!`
真人 B	`bidder_b@example.com`	`Test123456!`

README 必须给出可复制的完整路径：复制环境变量 → Compose 启动依赖 → 自动迁移与初始化 → 管理员开始演示拍卖 → 两个浏览器用户加入 → 启动 20 人模拟脚本 → 创建 Agent Token 并让一个竞拍 Agent 参与 → 查看成交、钱包和流水 → 运行自动化测试。

AI 协同开发交付规范

仓库根目录必须包含：

- `AI_USAGE.md`：使用了哪些 Agent 与模型、各模块人机分工的大致比例、至少三项本人设计决定及理由、至少两个未采用方案及原因、至少两个真实错误（现象、日志、定位、修复、验证），以及哪些代码目前仍不能独立解释或修改。
- `DESIGN.md`：前端、Solon 服务、MySQL、WebSocket 与 Agent API 的边界；状态机、出价事务与唯一结算；钱包与流水模型；并发方案及失败恢复；丢消息与重连策略。
- `DECISIONS.md`：至少三项决策，含背景、候选方案、最终选择、代价与验证结果。
- `DEBUG_LOG.md`：至少两个开发期间真实发生的问题，不接受事后编造。
- `AGENT_TOOL_SPEC.md`：说明评审如何向竞拍 Agent 提供 Token、查询拍卖、出价和读取结果。

同时保留有意义的开发历史，不接受最终一次性上传后再机械拆分提交，提交信息应说明行为变化（如 `feat: freeze wallet balance when bid becomes leader`）。另需提供 3～5 分钟录屏：真人加入、脚本用户并发、Agent 出价、最后五秒延时、结算与资金结果。视频不能替代代码、测试、

Git 与文档核验。

现场核验

通过初筛后进行两段式现场核验。第一段约 20 分钟，不使用 Coding Agent（允许查阅公开资料）：解释一次并发出价的事务过程、旧领先者资金如何释放、重复请求、重复结算和服务重启如何处理，并根据日志定位一个评审指定问题。第二段约 30～45 分钟，允许使用 Coding Agent 并共享屏幕，完成一个临时变更（例如 VIP 用户最小加价不同、取消拍卖后批量释放冻结金额、增加代理最高价、最大延长次数改为可配置并补迁移与测试）。评审关注您如何拆解任务、给 Agent 上下文、审查 diff、运行测试和控制修改范围，而不是 Agent 输出速度。

时限与提交

建议有效开发时间 8～12 小时，提交窗口为收到题目后 3 个自然日，允许分阶段交付。请提交：GitHub / Gitee 仓库地址及完整提交历史；3～5 分钟演示录屏；可访问的线上地址（加分项）；并将仓库地址、姓名、联系方式发送至 umqvqmu@163.com。

期待看到的不是一个被 Agent 包装得很大的项目，而是业务边界清楚、数据经得起并发验证、您能够真正理解并继续演进的作品。

评分标准（10 分）

维度	分值	主要关注点
架构与职责边界	1.5	Vue、Solon、MySQL、实时通信和 Agent API 的边界与取舍
拍卖业务与并发一致性	1.8	出价、冻结 / 释放、延时、幂等、唯一结算和失败恢复
Solon 后端能力	1.2	HTTP、WebSocket、鉴权、依赖注入、异常处理和模块组织
Vue 3 / TypeScript	1.0	类型、状态管理、实时交互、错误反馈和重连恢复
MySQL 实战能力	1.2	数据模型、事务、索引、约束、锁、流水和查询设计
代码质量	1.2	可读性、复杂度、职责划分、异常、安全和可维护性
测试与可验证性	0.8	单测、集成测试、并发脚本、断言质量和隐藏测试表现
Git 与交付规范	0.3	提交历史、README、环境变量、迁移、录屏和快速启动
Agent 使用能力	1.0	任务拆解、上下文组织、diff 审查、验证、纠错和责任边界
合计	10.0	

现场核验不能通过时，不因界面漂亮、代码量大或线上可访问而直接推进；最终结果以代码、测试、演示、Git 与现场核验的综合证据为准。